Evaluating Large Language Models Trained on Code (Codex)

M. Chen, J. Tworek, H. Jun, Q. Yuan, H. Ponde de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P. Mishkin, B. Chan, S. Gray, N. Ryder, M. Pavlov, A. Power, L. Kaiser, M. Bavarian, C. Winter, P. Tillet, F. Petroski Such, D. Cummings, M. Plappert, F. Chantzis, E. Barnes, A. Herbert-Voss, W. Hebgen Guss, A. Nichol, A. Paino, N. Tezak, J. Tang, I. Babuschkin, S. Balaji, S. Jain, W. Saunders, C. Hesse, A. N. Carr, J. Leike, J. Achiam, V. Misra, E. Morikawa, A. Radford, M. Knight, M. Brundage, M. Murati, K. Mayer, P. Welinder, B. McGrew, D. Amodei, S. McCandlish, I. Sutskever, W. Zaremba

arXiv (2021)

A language model for code

Program synthesis with language models

- Program synthesis is a longstanding challenge (Simon, 1963; Manna et al., 1971).
- Large code corpora became available, e.g. **CodeSearchNet** and **The Pile**.
- Self-supervised objectives used in NLP were adapted to code: **CodeBERT**, **T5**, and **PyMT5** are representative examples.
- Early analysis suggested that **GPT-3** could generate programs from Python docstrings even without explicit code-specialized training.
- Hypothesis of the paper: specializing a GPT language model for code could work across many tasks.

This work: Codex

Codex is a GPT-style model specialized for code. The paper positions it as a model that can generate useful code across tasks and notes that it is used for **Copilot**.

Refs: Chen et al. (2021); Husain et al. (2019); Gao et al. (2020); Feng et al. (2020); Brown et al. (2020); Wang et al. (2021).

Related Work

Learning to Execute

LSTMs are trained on programs and can learn algorithmic tasks such as adding two 9-digit numbers with about 99% accuracy. Curriculum learning plays a key role.

Naturalness of software

This work studies code with n-gram language models and shows that source code is statistically natural in ways that can be exploited for modeling and tooling.

SPoC

The SPoC dataset contains about 18K C++ programs and studies pseudocode-to-code synthesis using a seq2seq model with best-first search.

APPS

APPS introduces a benchmark with 10K programming problems across three difficulty levels and includes test cases for evaluating generated solutions.

Refs: Zaremba et al. (2014); Hindle et al. (2012); Kulal et al. (2019); Hendrycks et al. (2021).

Task and approach

Task: functions from docstrings

- Input: **docstring**
- Output: **Python function**
- Code correctness is evaluated automatically via **unit tests**.
- This differs from natural language generation, which usually needs human assessors or heuristic metrics.
- For benchmarking, the paper constructs **164 original programming problems** with tests.
- Problem types include language comprehension, algorithms, simple mathematics, and interview-style questions.

Problem solving with one sample

Approach: generate samples and check whether any pass the unit tests.

- Codex (12B): **28.8%**
- Codex (300M): **13.2%**
- GPT-J (6B): **11.4%**
- Other GPT models: approximately **0%**
- After supervised fine-tuning on correct implementations, Codex-S (12B) reaches **37.7%**

Problem solving with multiple samples

Programming usually involves iteration and bug fixing. Repeated sampling approximates this process.

- Codex-S (12B) with 100 samples solves **77.5%** of problems.
- Selecting the sample with highest mean log-probability solves **44.5%**.

Refs: Chen et al. (2021); Wang et al. (2021).

Evaluation framework

Functional correctness

- Match-based metrics compare against a reference exactly or fuzzily (e.g. BLEU).
- Such metrics are limited for code because many different programs can be equivalent.
- The paper therefore emphasizes **functional correctness**: a sample is correct if it passes a set of unit tests.
- This is also how human developers judge code in practice, e.g. via test-driven development and CI.

The `pass@k` metric

Generate k samples per problem. A problem is solved if *any* sample passes the tests.

Unbiased estimator used in the paper:

$$\texttt{pass@}k \triangleq \mathbb{E}_{\text{problems}}\left[1 - \frac{\binom{n-c}{k}}{\binom{n}{k}}\right]$$

where $n \geq k$ is the number of samples and c is the number of correct samples among them.
Advantage: using more generated samples reduces variance while keeping comparisons fair.

Refs: Chen et al. (2021); Papineni et al. (2002); Ren et al. (2020); Roziere et al. (2020); Kulal et al. (2019).

Evaluation details

HumanEval: hand-written evaluation set

- Functional correctness is evaluated on **164 hand-written problems**.
- Each HumanEval problem contains a function signature, a docstring, a body, and several unit tests (average: **7.7** tests per problem).
- Hand-written problems matter because the models are trained on GitHub, where many public benchmark solutions already exist.
- HumanEval targets simple mathematics, reasoning, and language comprehension.
- The dataset is made public for future benchmarking.

Sandbox for executing generated programs

- Executing public code and generated code creates a security risk.
- The paper therefore runs untrusted programs in a sandboxed environment.
- **gVisor** is used to isolate the host, and network-adjacent services are protected with eBPF-based firewall rules.

Refs: Chen et al. (2021); Hendrycks et al. (2021); OpenAI HumanEval repository; gVisor.

Nuances of pass@k estimation

Estimating `pass@k`

Aim: estimate the probability that among k samples, at least one is correct.

- If the true probability of a correct sample is p, then:

$$\Pr(\text{none correct}) = (1-p)^k, \qquad \mathrm{pass@}k = 1-(1-p)^k.$$

- A naive plug-in estimate $1-(1-\hat{p})^k$ from $\hat{p} = \mathrm{pass@}1$ systematically underestimates performance.
- It also makes results look better simply by drawing more samples from the same pool.
- The paper's estimator instead compares tasks across different numbers of samples in a consistent way.

Figure placeholder

Insert the two estimator-comparison plots from the original slide here.

Top: $1-(1-\hat{p})^k$ estimator
Bottom: unbiased combinatorial estimator

Key point

Slightly higher initial variance is acceptable because the estimator enables fairer comparisons across different sample budgets.

Refs: Chen et al. (2021).

Nuances of pass@k estimation

Why the proposed estimator is unbiased

The second term in

$$\mathbb{E}\left[1 - \frac{\binom{n-c}{k}}{\binom{n}{k}}\right]$$

directly estimates the probability of drawing **only failed samples** when choosing k items without replacement from n candidates.

Setup

- Let $c \sim \mathrm{Binom}(n, p)$, where p is $\mathrm{pass@1}$.
- If $n - c < k$, then $\binom{n-c}{k} = 0$, so the estimator returns 1.
- Otherwise, the estimator computes the probability that all k chosen samples are failures.

Sketch of the derivation

$$\mathbb{E}\left[\frac{\binom{n-c}{k}}{\binom{n}{k}}\right] = \sum_{i=0}^{n-k} \frac{\binom{n-i}{k}}{\binom{n}{k}} \binom{n}{i} p^i (1-p)^{n-i}$$

$$= (1-p)^k \sum_{i=0}^{n-k} \binom{n-k}{i} p^i (1-p)^{n-k-i} = (1-p)^k$$

Therefore,

$$\mathbb{E}[\mathrm{estimator}] = 1 - (1-p)^k = \mathrm{pass@}k.$$

Refs: Chen et al. (2021).

Implementing the pass@k estimator

Numerically stable implementation

```
def pass_at_k(n, c, k):
    """
    n: total number of samples
    c: number of correct samples
    k: k in pass@k
    """
    if n - c < k:
        return 1.0
    return 1.0 - np.prod(
        1.0 - k / np.arange(n - c + 1, n + 1)
    )
```

Rationale for $n - c < k$

If fewer than k failed samples exist, then it is impossible to draw k failures without replacement:

$$\binom{n-c}{k} = 0 \quad \Rightarrow \quad 1 - \frac{\binom{n-c}{k}}{\binom{n}{k}} = 1.$$

Rationale for $n - c \geq k$

Rewrite the combinatorial ratio as a stable product:

$$\frac{\binom{n-c}{k}}{\binom{n}{k}} = \prod_{j=n-c+1}^{n} \left(1 - \frac{k}{j}\right)$$

which is exactly what the NumPy implementation computes.

Refs: Chen et al. (2021).

Code Fine-tuning

Overview

Codex is produced by fine-tuning GPT models of up to **12B** parameters. Unlike GPT models trained only on text, Codex obtains non-trivial HumanEval performance, and with 100 samples per problem, the majority of problems have at least one solution.

Data collection and fine-tuning

- Training corpus collected in May 2020 from **54M GitHub repositories**.
- Produced **179 GB** of unique Python files under 1 MB.
- Filtering removed likely auto-generated files, very long lines, and files with a very small fraction of alphanumerics.
- Final result: **159 GB** of unique Python files.
- Surprisingly, fine-tuning from GPT-3 gave no improvement over training from scratch, although it converged faster and was used in practice.

Optimization and tokenization

- Same learning rate as the corresponding GPT model.
- Linear warmup for 175 steps; cosine decay afterwards.
- Training on **100B tokens** with Adam ($\beta_1 = 0.9,\ \beta_2 = 0.95,\ \epsilon = 10^{-8}$) and weight decay 0.1.
- GPT-3 tokenization is reused, but extra whitespace tokens are added for code.
- This reduces the token count for code by roughly **30%**.

Refs: Chen et al. (2021); Kingma and Ba (2015).

Supervised Fine-tuning

Overview

GitHub data contains far more than clean “function from docstring” examples: it also includes classes, config files, scripts, and data files. This mismatch may limit HumanEval performance, so the paper builds a supervised training set of standalone function-completion tasks.

Source 1: competitive programming problems

- Problems, solutions, and function signatures were collected from interview-preparation and programming-contest websites.
- Problem descriptions are used as docstrings.
- Unit tests are created from examples in statements and by submitting incorrect solutions to reveal hidden cases.
- These sources provide self-contained tasks with strong algorithmic content and usually good test coverage.
- Roughly **10,000** problems are curated from these sources.

Refs: Chen et al. (2021).

Supervised Fine-tuning

Source 2: problems from Continuous Integration

- Problems are also extracted from open-source repositories by tracing inputs and outputs during integration tests with `sys.setprofile`.
- CI config files reveal how to create environments, install dependencies, and run tests.
- Repositories using **Travis** or **Tox** are particularly useful; additional source is pulled from PyPI.
- Because the code is untrusted, all integration tests are executed in the sandbox.
- Only about **40,000** problems are collected from millions of functions, mainly because not all functions are easy to trace and some runtime objects cannot be restored outside the sandbox.

Filtering problems

- Automatically gathered prompts may be ambiguous or stateful.
- Codex-12B is used to generate 100 samples per problem.
- If all samples fail, the problem is discarded as too hard or ambiguous.
- Verification is rerun multiple times to filter out stateful problems.

Refs: Chen et al. (2021); Travis CI; Tox.

Supervised Fine-tuning

Methodology - training with prompts

Training examples are assembled in the same prompt format used at evaluation time.

```
def solution(lst):
    """Given a non-empty list of integers, return the sum
    of all odd elements that are in even positions.

    Examples
    solution([5, 8, 7, 1]) => 12
    solution([3, 3, 3, 3, 3]) => 9
    solution([30, 13, 24, 321]) => 0
    """
    return sum(lst[i] for i in range(0, len(lst))
               if i % 2 == 0 and lst[i] % 2 == 1)
```

- Codex is fine-tuned on these training problems to produce **Codex-S**.
- Training minimizes the negative log-likelihood of the reference solution while masking the prompt.
- Shorter prompts are left-padded so the target solutions line up.
- Learning rate is 1/10 of Codex with the same schedule until validation loss plateaus (after fewer than 10B tokens).

Optimal temperatures

- Codex-S prefers higher temperatures than Codex for all cases with $k > 1$.
- The paper suggests this may reflect a narrower learned distribution.
- For later evaluation:
  - $T^* = 0$ for `pass@1`
  - $T^* = 1$ for `pass@100`

Insert the “best temperature for different k” plot here
(Codex vs. Codex-S)

Refs: Chen et al. (2021).

Comparative Analysis of Related Models

Related approaches

- GPT-Neo and GPT-J-6B are the closest comparison points.
- Both are trained on The Pile, of which about 8% comes from GitHub.
- GPT-J-6B already produces qualitatively reasonable code.

Temperatures

GPT-Neo: 0.2, 0.4, 0.8
GPT-J-6B: 0.2, 0.8
Tabnine: 0.4, 0.8

HumanEval results

Model	$k = 1$	$k = 10$	$k = 100$
GPT-Neo 125M	0.75%	1.88%	2.97%
GPT-Neo 1.3B	4.79%	7.47%	16.30%
GPT-Neo 2.7B	6.41%	11.27%	21.37%
GPT-J 6B	11.62%	15.74%	27.74%
Tabnine	2.58%	4.35%	7.59%
Codex-12M	2.00%	3.62%	8.58%
Codex-25M	3.21%	7.10%	12.89%
Codex-42M	5.06%	8.80%	15.55%
Codex-85M	8.22%	12.81%	22.40%
Codex-300M	13.17%	20.37%	36.27%
Codex-679M	16.22%	25.70%	40.95%
Codex-2.5B	21.36%	35.42%	59.50%
Codex-12B	28.81%	46.81%	72.31%

Takeaway

Codex-12B goes considerably beyond the performance of prior models.

Parameter note

The slide emphasizes that Codex-12B achieves this despite having roughly **20x fewer parameters than GPT-J-6B**.

Refs: Chen et al. (2021); Black et al. (2021); Wang et al. (2021); Gao et al. (2020).

Results on the APPS Dataset

APPS dataset comparison

- APPS was introduced to measure coding challenge competence.
- It contains **5000 train** and **5000 test** coding problems.
- Each example includes unit tests; training examples also include solutions.
- Most APPS tasks are **full-program synthesis** problems (stdin/stdout), not single-function generation.
- Original APPS metrics: **strict accuracy** and **test case average**.
- Codex results here are reported only under strict accuracy / `pass@k`.

Implementation details and results

Additional factors on the slide:

- Example cases: APPS gives 3 I/O examples; Codex uses one example as a formatting hint (“1-shot”).
- Timeouts: results are reported both with and without counting solutions that pass tests but time out after 3 seconds.

Method	Intro	Interview	Competition
GPT-Neo 2.7B raw `p@1`	3.90	0.57	0.00
GPT-Neo 2.7B raw `p@5`	5.50	0.80	0.00
1-shot Codex raw `p@1`	4.14 (4.33)	0.14 (0.30)	0.02 (0.03)
1-shot Codex raw `p@5`	9.65 (10.05)	0.51 (1.02)	0.09 (0.16)
1-shot Codex raw `p@100`	20.20 (21.57)	2.04 (3.99)	1.05 (1.73)
1-shot Codex raw `p@1000`	25.02 (27.77)	3.70 (7.94)	2.33 (5.85)
1-shot Codex filtered `p@1`	22.78 (25.10)	2.64 (5.78)	3.04 (5.25)
1-shot Codex filtered `p@5`	24.52 (27.15)	3.23 (7.13)	3.08 (5.53)

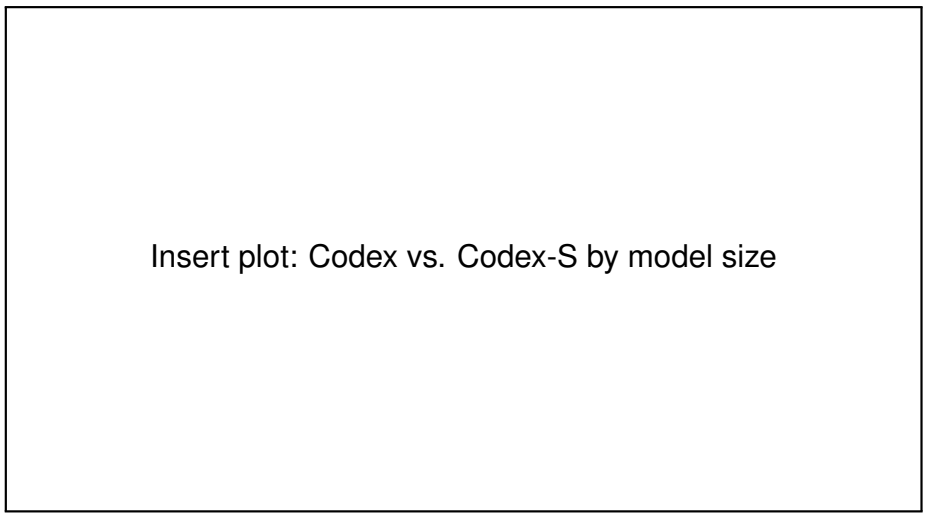
Temperature 0.6 is used for all k in `pass@k`.

Refs: Chen et al. (2021); Hendrycks et al. (2021); Black et al. (2021).

Supervised Fine-tuning: Results

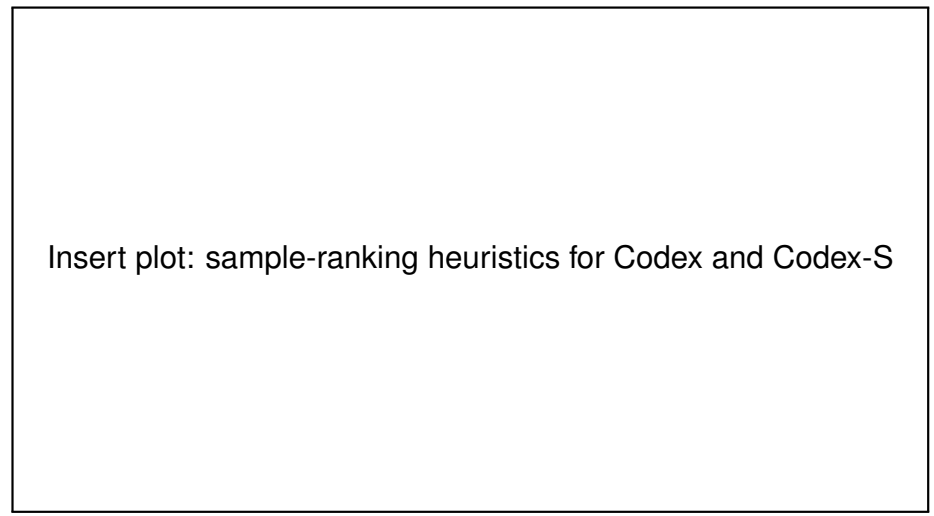
Influence of model size

- Codex-S beats Codex by an average of **6.5%** on `pass@1`.
- On `pass@100`, the average gain is **15.1%**.
- The slide shows both curves increasing with model size.



Sample ranking heuristics

- Oracle reranking remains an upper bound.
- Mean log-probability is again the best practical heuristic.
- The average benefit of mean log-probability is about **2% higher** for Codex-S than for Codex.



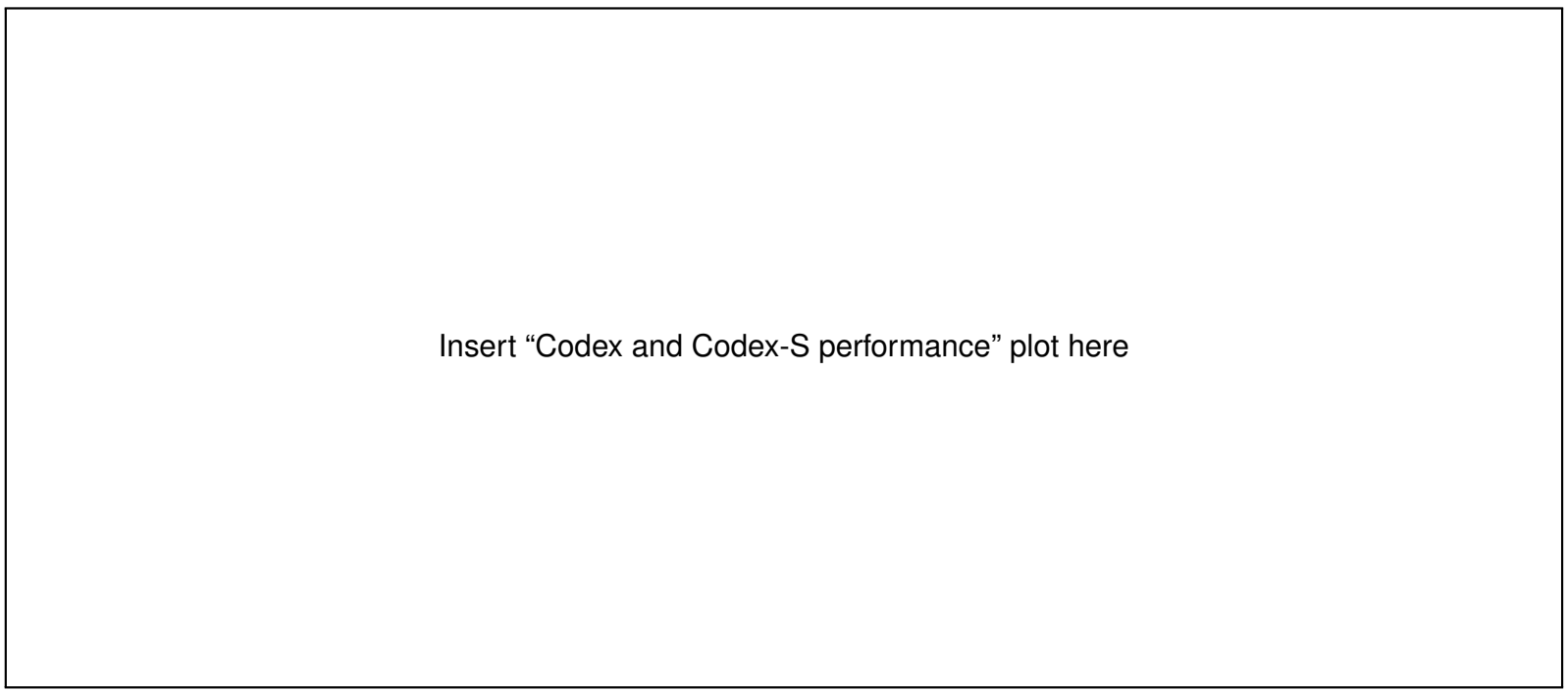
Refs: Chen et al. (2021).

Comparing Codex and Codex-S

Comparing training strategies on HumanEval

The original slide shows a single scaling plot across model sizes for GPT-3, Codex, and Codex-S, together with reranking variants.

- **Codex-S mean-logp reranking** solves **44.5%** when reranking 100 samples.
- **Codex-S oracle reranking** solves **77.5%** when selecting from 100 samples using the unit tests.



Refs: Chen et al. (2021).

Docstring complexity

Composing building blocks

The 13 building blocks are chained by concatenating:

- their one-line descriptions into a docstring, and
- their one-line implementations into a code body.

Example from the slide:

```
def string_manipulation(s: str):
    """
    This function takes a string as input, then
    returns the result of performing the following
    sequence of manipulations:
    - make every other character uppercase
    - replace spaces with triple spaces
    """
    s = "".join(char.upper() if i % 2 == 0 else char
                for i, char in enumerate(s))
    s = s.replace(" ", "   ")
    return s
```

Results

Insert “Synthetic pass rate vs. components” plot here

- As each component is added, the pass rate drops by about **2–3x**.
- Human programmers do not show the same degradation once they can chain two operations.
- Codex also makes variable-binding mistakes when many variables are mentioned.

Example binding failure

Prompt idea: “Add 3 to `y`, then subtract 4 from both `x` and `w`. Return the product of the four numbers.”
Observed failure: the generated code subtracts 4 from `x` but forgets to subtract 4 from `w`, and only multiplies two numbers.

Refs: Chen et al. (2021).

Broader Impacts and Hazard Analysis

Applications of Codex

Potentially useful applications listed on the slide:

- onboarding users to new codebases,
- reducing context switches for experienced coders,
- enabling non-programmers to write specifications,
- producing draft implementations,
- aiding education and exploratory coding.

Hazard analysis and over-reliance

- Main risks: generated code may not match user intent, and the system may be misused.
- The analysis is framed in terms of risk factors rather than full product safety design.
- Over-reliance is a central concern, especially for novice programmers.
- Generated code may look correct but still be wrong or insecure.
- Automation bias makes this especially dangerous: people often trust automated suggestions too much.
- Human oversight remains necessary.

Refs: Chen et al. (2021); Leveson (2019).

Summary

Summary

- The paper studies whether large language models can be trained to generate code from docstrings.
- After GitHub fine-tuning, Codex performs well on human-written problems, especially easy interview-style tasks.
- Performance improves further when training is brought closer to the evaluation distribution and when multiple samples are considered.
- Codex-D is introduced for code-to-docstring generation; it is weaker than code generation but still comparable.
- The paper also emphasizes limitations, broader impacts, and safety risks such as over-reliance, bias, misalignment, insecurity, and economic change.

Refs: Chen et al. (2021).

Backup / Q&A

Prompting for evaluation

Prompting to compute `pass@k`

Each HumanEval problem is assembled into a prompt made from the function signature, the docstring, and any examples.

```
def incr_list(l: list):
    """Return list with elements incremented by 1.
    >>> incr_list([1, 2, 3])
    [2, 3, 4]
    >>> incr_list([5, 3, 5, 2, 3, 3, 9, 0, 123])
    [6, 4, 6, 3, 4, 4, 10, 1, 124]
    """
```

Sampling continues until one of the following stop tokens is encountered: `'\nclass'`, `'\ndef'`, `'\n#'`, `'\nif'`, or `'\nprint'`.

Sampling details

- Nucleus sampling with $p = 0.95$ is used for all sampling evaluation.
- For the simple single-function prompt on the slide, Codex 12B reaches about **pass@1 = 0.9**.

Multi-function prompts

The paper notes a dramatic degradation when the prompt contains multiple functions. In the example on the slide, Codex 12B gets only about **pass@1 = 0.005**.

Refs: Chen et al. (2021); Holtzman et al. (2019).

Loss scaling and temperature

Loss scaling

- Language model losses often follow a power law.
- The paper finds a similar pattern for Codex test loss on a held-out GitHub validation set.
- Approximate scaling law shown on the slide:

$$\left(\frac{N}{5.92 \times 10^7}\right)^{-0.13}$$

- Takeaway: Codex fine-tuning appears to scale smoothly with model size.

Insert loss-scaling plot here

Sampling temperature

- Best temperature depends on k.
- For larger k, higher temperatures work better because `pass@k` rewards whether the model generates *any* correct solution.
- The slide shows both `pass@k` vs. k for several temperatures and a curve for the best temperature as a function of k.

Insert temperature-vs-`pass@k` plots here

Refs: Kaplan et al. (2020); Chen et al. (2021).

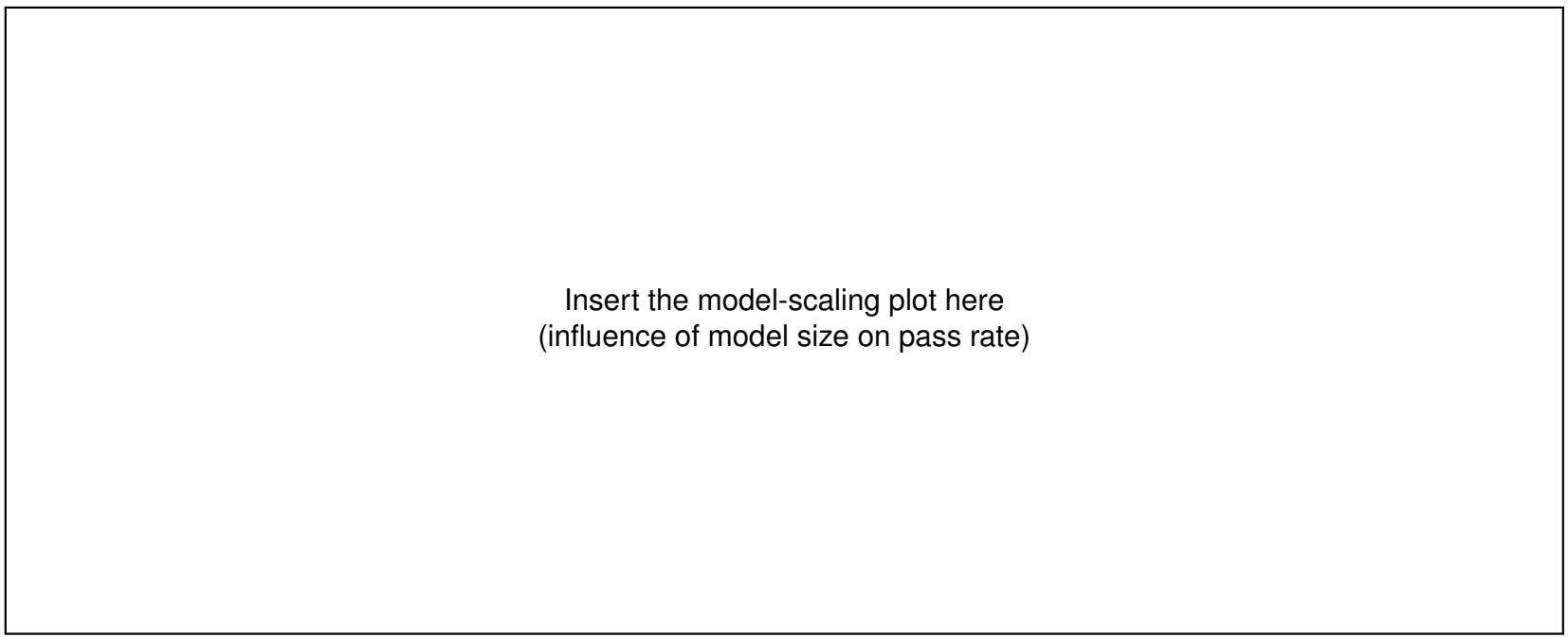
Model scaling at optimal temperatures

Model scaling

For a **679M**-parameter model, the best temperatures reported on the slide are:

- $T^* = 0.2$ for `pass@1`
- $T^* = 0.8$ for `pass@100`

The main takeaway is that performance scales smoothly as a function of model size.

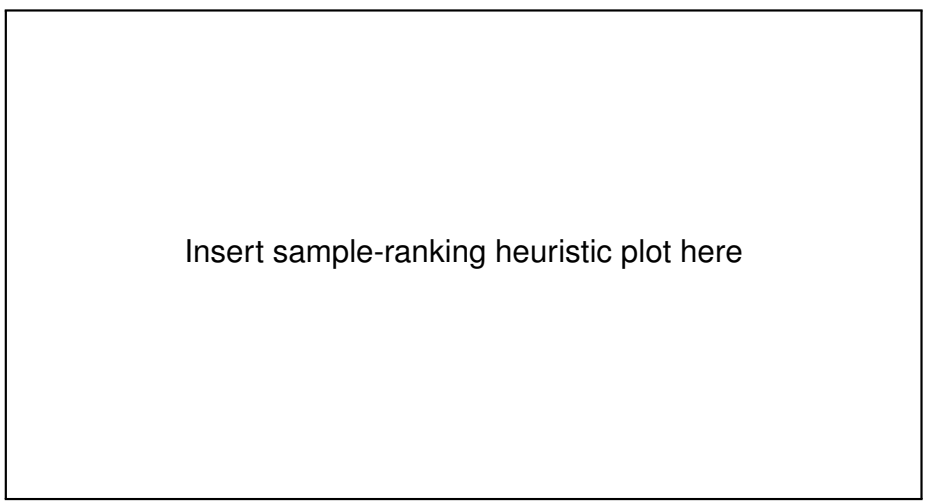


Refs: Chen et al. (2021).

Sampling heuristics and BLEU score

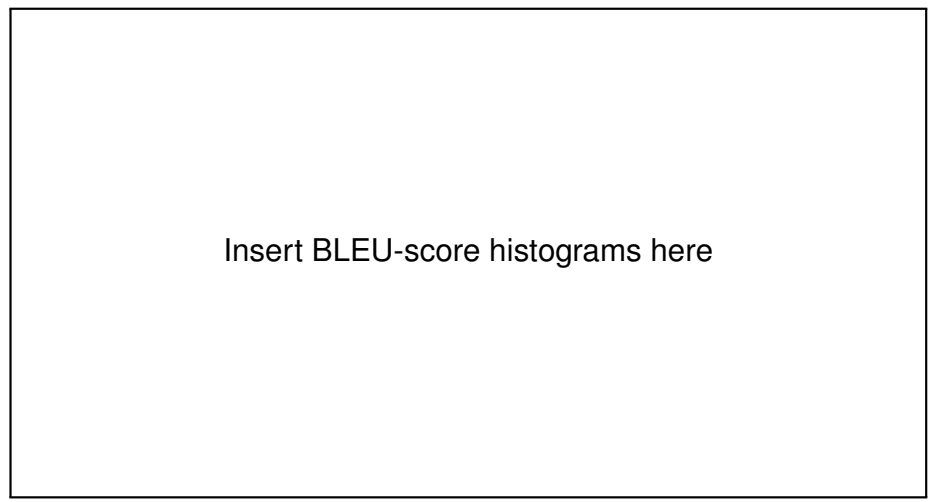
Effectiveness of sampling heuristics

- `pass@k` can be interpreted as evaluating the best out of k samples.
- An oracle that knows the unit tests is the upper bound.
- Without an oracle, a practical goal is to select one sample among k candidates.
- For Codex 12B at $T = 0.8$, the best non-oracle heuristic is **mean log-probability**.



BLEU score correlation

- BLEU scores are computed for HumanEval Codex 12B samples at $T = 0.8$.
- The comparison is made against reference solutions.
- Correct and wrong solutions are **not cleanly separable** by BLEU.



Refs: Chen et al. (2021).

Code generation examples

Prompt

```
def is_prime(n):
    """Return true if a given number is prime.
    >>> is_prime(6)
    False
    >>> is_prime(101)
    True
    >>> is_prime(4)
    False
    >>> is_prime(1)
    False
    """
```

8 random samples are shown on the original slide (temperature 0.8).

Completions 1–3 (wrong)

- **Completion 1:**

```
for i in range(2, n):
    if n % i == 0:
        return False
return True
```

  Fails for $n = 1$ because it returns True.

- **Completion 2:** exactly the same as Completion 1.
- **Completion 3:**

```
# TODO: implement this function
pass
```

  Fails all unit tests.

Completions 4–5

- **Completion 4 (correct):** handles $n < 2$, handles 2, rejects evens, and tests odd divisors up to $\sqrt{n}$.
- **Completion 5 (wrong):** uses a helper to test divisibility up to $\sqrt{n}$, but still returns True for $n = 1$.

Refs: Chen et al. (2021).

Code generation examples

Completions 6–8

- **Completion 6 (correct):**

```
prime = True
if n == 1:
    return False
for i in range(2, n):
    if n % i == 0:
        prime = False
return prime
```

  Exhaustive check, but correct.

- **Completion 7 (wrong):** repeats the same bug as Completion 1.
- **Completion 8 (correct):** efficient primality test based on checking divisibility by 2, 3, and then numbers of the form $6k \pm 1$.

Why $6k \pm 1$ matters

For primes greater than 3, any candidate factor must be of the form $6k \pm 1$.

- Numbers of the form $6k$, $6k + 2$, and $6k + 4$ are divisible by 2.
- Numbers of the form $6k + 3$ are divisible by 3.
- So after handling 2 and 3, only $6k - 1$ and $6k + 1$ remain worth testing.

This makes Completion 8 faster than checking every number up to n.

Refs: Chen et al. (2021); standard primality-test observation.